Practical File
of
Internet of Things (IoT)

Subject Code – 24CAP-751

Submitted to

Mr. Sachin Chawla

Assistant Professor

UIC Department

Submitted By

Prince

24MCA20451

24MCA-2B

UNIVERSITY INSTITUTE OF ENGINEERING CHANDIGARH UNIVERSITY GHARUAN, MOHALI, PUNJAB, INDIA-140413

Apr-2026

INDEX

Sno	Name of Experiment	Date	Conduct (12 marks)	WS (8 marks)	Viva (10 marks)	Marks	Total marks (30)	Sign.
1	Design first sketch to interface Led with Arduino Uno and to design different patterns.	17/01						
2	Design the circuit and WAP to interface an ultrasonic sensor (SR-04) with Arduino Uno to measure distance and print on LCD.	02/02						
3	Design the circuit and WAP to interface the Arduino with RFID card and LCD	24/02						
4	Design the circuit and WAP to interface the DHT-11 Sensor with Arduino uno	03/04						
5	Design the circuit and WAP to interface the Node MCU with Led and Buzzer. Control Led and buzzer using Blynk cloud	13/04						
6	Design the circuit and WAP to interface Temperature Sensor (LM36) or DHT11 with Arduino Uno and monitor the sensor reading via Blynk cloud	13/04						
7	Minor Project – Iot Nexus Automation							

Project Report

Student Name: Prince **UID:24MCA20451**
Branch: UIC/MCA **Section/Group: 2B**
Semester: 4th **Date of Performance:18/04/26**
Subject Name: IOT LAB **Subject Code: 24CAP-751**

1. Aim

To design and develop an IoT-based room monitoring and alert system using the ESP8266 NodeMCU that can detect human presence and monitor ambient temperature in real time.

The system provides local alerts through LEDs and a buzzer, and sends remote notifications to the user's smartphone via the Blynk IoT platform whenever room temperature exceeds a predefined threshold.

2. Objectives

- Interface a DHT11 temperature sensor to continuously read and monitor ambient room temperature.
- Trigger a Red LED and active buzzer as local alerts when temperature crosses the preset threshold of 30°C.
- Push remote smartphone notifications via Blynk app when abnormal conditions are detected.
- Demonstrate a complete end-to-end IoT system: sensing → processing → local actuation → cloud reporting.

3. Components Required:

S.No	Component Name	Quantity
1	NodeMCU (ESP8266 V3)	1
2	PIR Motion Sensor (HC-SR501)	1
3	DHT11 Sensor	1
4	Green LED	1
5	Red LED	1
6	220Ω Resistor	2
7	Active Piezo Buzzer	1
8	Breadboard	1
9	Jumper Wires	As required
10	Blynk IoT Platform	—

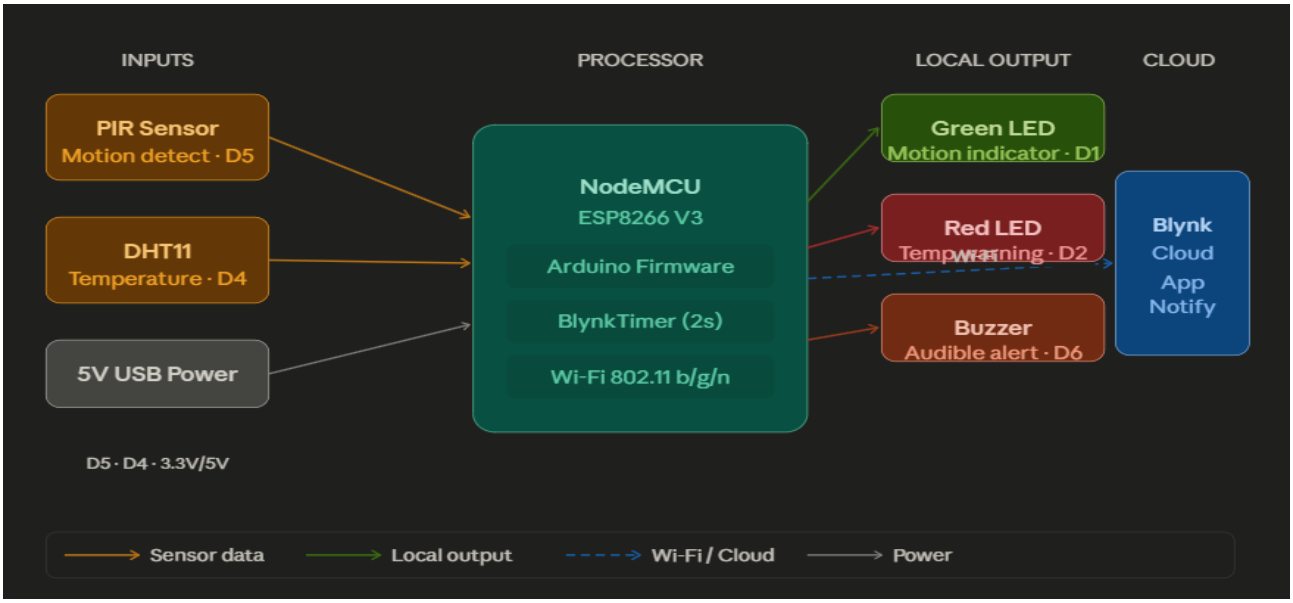
4. Details of Components:

- NodeMCU ESP8266 V3 — The heart of this system. It runs the Arduino-compatible C++ firmware, reads sensors via GPIO pins, executes the alert logic, and simultaneously maintains a TCP connection to the Blynk cloud over Wi-Fi. The BlynkTimer library ensures non-blocking 2-second polling without using delay().
- PIR Motion Sensor (HC-SR501) — Uses passive infrared detection. It measures infrared radiation emitted by warm bodies (humans). When a person enters the detection cone, the sensor output goes HIGH. It requires a ~30–60 second warm-up after power-on to stabilize and avoid false positives. Connected to D5; powered at 5V/VIN.
- DHT11 Temperature Sensor — A capacitive humidity/temperature sensor. Only temperature readings are used here. It communicates over a single data line using a

custom 1-wire-like protocol. The DHT library handles timing internally. Powered at 3.3V (important — not 5V). Has ±2°C accuracy and 1Hz sampling rate.

- LEDs and Resistors — The Green LED (D1) gives instant visual feedback when motion is detected. The Red LED (D2) glows when temperature exceeds 30°C. The 220Ω resistors limit forward current to a safe 10–15 mA, preventing GPIO pin damage on the NodeMCU.
- Active Piezo Buzzer — Unlike a passive buzzer, this type has an internal oscillator. Applying 3.3V HIGH from pin D6 makes it emit a continuous tone without needing a PWM signal. It activates synchronously with the Red LED as part of the high-temperature alert.
- Blynk IoT Platform — A cloud service that bridges the NodeMCU and the user's smartphone. Virtual pin V0 carries motion status (0/1), V1 carries temperature float value, and V2 carries a status string. Blynk.logEvent() triggers a push notification to the mobile app when high temperature is detected.

5. Block Diagram of Designed Model:



6. Working of Designed Model:

- Power-up & Initialization

  NodeMCU boots, initializes GPIO pins (PIR as INPUT; LEDs and Buzzer as OUTPUT), starts DHT11 library, and connects to the configured Wi-Fi network. Blynk.begin() establishes a persistent TCP connection to the Blynk cloud server.

- Periodic Sensor Polling (every 2 seconds)

  BlynkTimer fires sendSensorData() every 2000ms. This function reads digitalRead(D5) for motion state and dht.readTemperature() for temperature. The isnan() guard discards any failed DHT reads.

- Motion Detection Logic

  If PIR output is HIGH → Green LED turns ON and Blynk.virtualWrite(V0, 1) updates the app dashboard. If no motion → Green LED turns OFF and V0 is set to 0.

- Temperature Monitoring & Alert

  Temperature value is always sent to Blynk via virtualWrite(V1). If temperature > 30°C: Red LED ON, Buzzer ON, logEvent("high_temp_alert") triggers a push notification to the smartphone, and V2 shows "High Temperature Alert!". Below threshold, all alerts reset.

- Continuous Loop

  Blynk.run() keeps the cloud connection alive. timer.run() manages all scheduled callbacks. The system runs indefinitely, updating every 2 seconds without blocking.

7. Pictures of Prototype:

Fig: Designed Prototype

8. Output of Deigned Model:

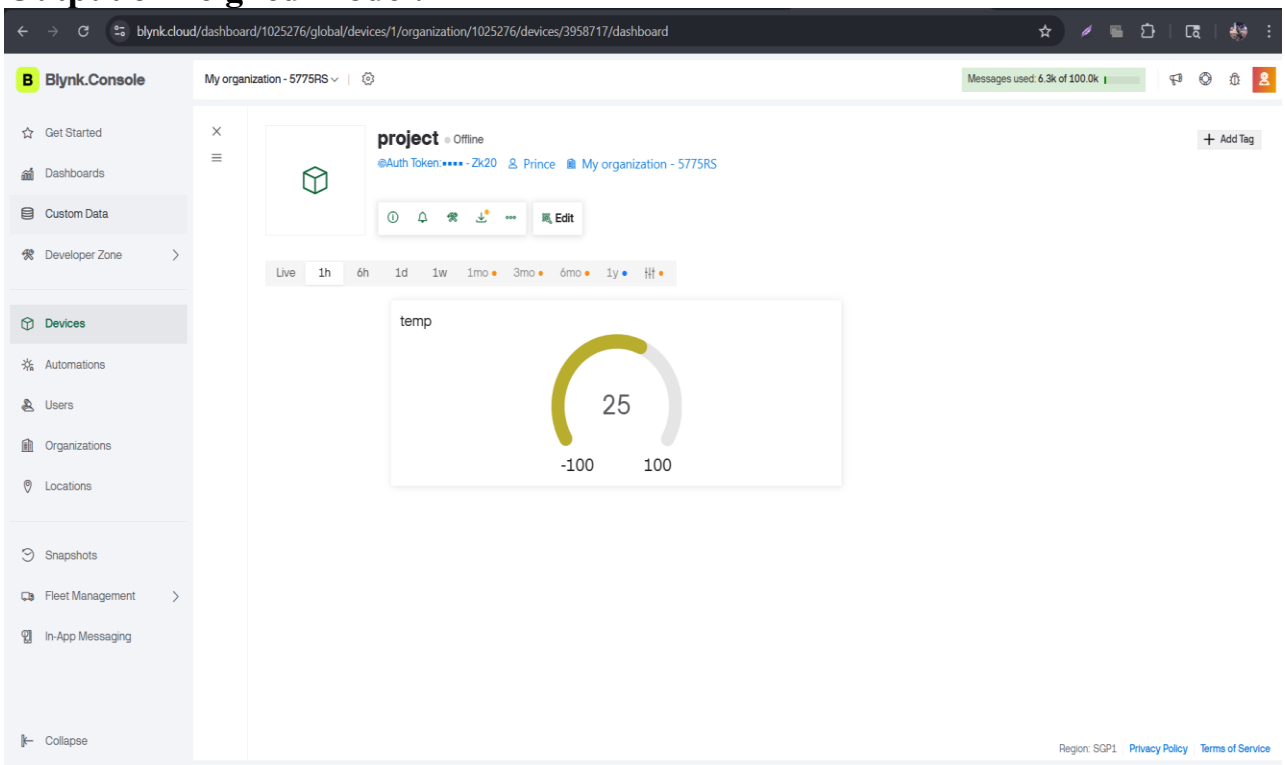


Fig: Output on Blynk cloud

9. Learning outcomes (What I have learnt):

- Multi-sensor interfacing: Learnt how to connect and read from two different types of sensors simultaneously — a digital PIR sensor at 5V and a single-wire DHT11 at 3.3V — on the same microcontroller without conflicts.
- Wi-Fi-enabled microcontrollers: Understood how ESP8266 integrates both a microcontroller and a Wi-Fi transceiver, enabling IoT connectivity without external modules. Learnt to configure and use the ESP8266WiFi library.

- Blynk IoT cloud integration: Gained practical experience in setting up a Blynk project, configuring virtual pins, sending real-time data using virtualWrite(), and triggering push notifications with logEvent() to a smartphone app.
- Non-blocking embedded programming: Learnt the importance of avoiding delay() in IoT firmware. Used BlynkTimer's setInterval() to schedule sensor polling every 2 seconds without halting the main loop, keeping Blynk connectivity alive.
- Alert system design: Learnt to design a dual-layer alert system — local hardware alerts (LED + buzzer) for immediate in-room response and remote cloud alerts (Blynk push notification) for user notification when away, covering both presence and absence scenarios.
- Real-time environmental monitoring: Understood how IoT devices can continuously monitor physical parameters (temperature, motion) and stream the data to a cloud dashboard, forming the basis of smart home and industrial monitoring systems.
- Debugging & Arduino IDE skills: Improved practical skills in using Serial Monitor for debugging sensor values and connection status, managing library installations via Library Manager, and selecting the correct board and COM port for uploading firmware.